



MicroGCC

Entrega Final

v1.0.0

1. Introducción	2
2. Modelo Computacional	3
2.1. Dominio (General Purpose Language):	3
2.2. Lenguaje	3
3. Implementación	4
3.1. Frontend	4
3.2. Backend	5
3.3. Adicionales	6
3.4. Dificultades encontradas	6
4. Futuras extensiones	7
5. Conclusiones	7
6. Referencias	8

Integrantes

Nombre	Apellido	Legajo	E-mail
Luciano	Stupnik	64233	lstupnik@itba.edu.ar
Matias	Rossi Seifert	63202	mrossiseifert@itba.edu.ar
Valentino	Sanguinetti	63098	vsanguinetti@itba.edu.ar
Junior	Rambau	64461	jrambau@itba.edu.ar

1. Introducción

MicroGCC es un compilador simplificado inspirado en el lenguaje de programación C. Está diseñado para traducir un subconjunto representativo de instrucciones y estructuras del lenguaje C a código ensamblador (ASM) para arquitecturas de 64 bits (x86-64, sintaxis Intel).

Entre sus características se incluyen el soporte para asignaciones, expresiones aritméticas y lógicas, estructuras condicionales, bucles, definición de funciones, declaración de variables y manejo de llamadas entre funciones.

El lenguaje fuente utilizado por MicroGCC es una variante simplificada de C, centrada en construcciones fundamentales de la programación imperativa, lo que permite una traducción directa a instrucciones de bajo nivel.

2. Modelo Computacional

2.1. Dominio:

El dominio de *MicroGCC* se sitúa en la intersección entre los lenguajes de alto nivel y la arquitectura de hardware de intel x86-64. El sistema fue diseñado para traducir un subconjunto bien definido de C a código ensamblador Intel-syntax de 64 bits. De esta manera, se puede compilar un lenguaje de propósito general, a las instrucciones necesarias en el lenguaje ensamblador del procesador de intel x86-64 para ser re-compilado (utilizando herramientas como *nasm* y *ld*) a un programa ejecutable.

2.2. Lenguaje

El lenguaje ofrece las siguientes funcionalidades:

- **Declaración de variables de tipos primitivos:** Se permite declarar variables con o sin inicialización, como `'int x = 5;'`. Se soportan tipos *int* y *char*.
- **Arrays:** Se permite la declaración de arreglos unidimensionales de tamaño fijo. Pueden ser utilizados en asignaciones y también enviados como parámetro a funciones. La siguiente instrucción `'char x[10];'` declara un array de chars, el cual contiene 10 elementos, de la misma forma que se hace en C.
- **Variables globales:** Se admite la declaración de variables globales. No pueden realizarse llamadas a funciones desde una asignación a una variable global.
- **Asignaciones:** Se permite asignar valores a variables ya declaradas, incluyendo asignaciones dentro de expresiones, como por ejemplo `'while ((a = a + 1) > 1);'`.
- **Operaciones aritméticas y lógicas:** Se soportan operaciones básicas como suma (+), resta (-), multiplicación (*), división (/) y módulo (%), así como operadores lógicos y relacionales como la igualdad (==), desigualdad (!=) y negación (!).
- **Estructuras condicionales:** Se implementa el soporte para condicionales como `'if'` y `'if-else'` permitiendo bloques de código solo para ciertos escenarios.

- **Bucles:** Se permite el uso de bucles como 'while' y 'for' que facilitan la repetición de código en situaciones necesarias
- **Funciones:** Se permite declarar y definir funciones con parámetros (tanto variables como arreglos) y valores de retorno. Las funciones pueden ser llamadas desde cualquier punto del programa, excepto dentro de una asignación a una variable global. También se permite declarar funciones externas, para luego ser compiladas desde otro archivo.
- **Comentarios:** Se admite el uso de comentarios tanto de una sola línea (//) como multilínea (/* ... */) para mejorar la documentación del código.
- **Generación de código Assembly:** Todo el código escrito en este lenguaje acaba siendo traducido a instrucciones en lenguaje ensamblador de 64 bits respetando la semántica del programa

3. Implementación

3.1. Frontend

El frontend de nuestro compilador se encarga de procesar el código fuente en C y transformarlo en una representación estructurada que pueda ser analizada y convertida a código assembler. Para esto utilizamos **Flex** para el análisis léxico y **Bison** para el análisis sintáctico.

El análisis léxico identifica los tokens del lenguaje. Estos incluyen los identificadores (nombres de variables y funciones), los tipos de datos, las operaciones aritméticas, las asignaciones, entre otras cosas. Luego, Bison se encarga de aplicar las reglas gramaticales definidas en el archivo de gramática y construir el árbol de sintaxis abstracta (AST), que representa la estructura lógica del programa.

Durante el diseño de la gramática nos encontramos con varias dificultades, sobre todo por la forma en que se estructura el lenguaje C. El subset de C incluye muchas construcciones que comienzan y terminan de forma similar, como por ejemplo, la declaración de funciones o de variables, lo cual generaba conflictos frecuentes de tipo *shift/reduce* o *reduce/reduce* en Bison.

3.2. Backend

Una vez construido el árbol de sintaxis abstracta (AST), se pasa al análisis semántico. Esta etapa está a cargo del módulo *SemanticAnalyzer*, que recorre el árbol y valida que el programa tenga sentido desde el punto de vista semántico.

Entre otras cosas, se asegura de que no haya variables re-declaradas en un mismo *scope*, que las asignaciones y operaciones respeten el tipo de dato requerido, que las funciones estén correctamente definidas, que sus llamadas respeten la cantidad y tipo de parámetros esperados, entre otras cosas. Esto permite detectar errores que no podrían encontrarse solo con la gramática.

Durante este proceso, se completa la tabla de símbolos (*SymbolTable*), donde se almacena la información de todas las variables y funciones del programa, organizadas por scopes. Cada función tiene su propio scope (desde su declaración hasta el final de su cuerpo), mientras que el scope global contiene las definiciones de funciones y variables globales. Estas últimas (las variables) se almacenan luego en la sección *.bss* del archivo assembler.

La tabla de símbolos incluye, además del tipo y nombre de cada entidad, el offset relativo al registro *RBP* que permite acceder a variables locales y parámetros en tiempo de ejecución, una vez armado el stack frame correspondiente. Este offset se asigna durante el análisis semántico y se consulta luego en la etapa de generación de código. En este proceso nos fijamos siempre en mantener el stack alineado. Hay una única tabla de símbolos y el scope se indica con el campo *functionName*, donde NULL indica que el scope en el que se encuentra el identificador es global.

Una vez finalizado el análisis semántico, se invoca al módulo *Generator*, que recorre el AST y genera el código assembler correspondiente. Este módulo no modifica la tabla de símbolos, sino que la consulta constantemente para obtener los datos necesarios:

- Para variables locales y parámetros, usa el offset relativo a *RBP* luego de haber armado el stack frame.
- Para variables globales, se utiliza directamente su nombre (ya que el análisis semántico garantiza que serán únicos) como referencia para acceder a la sección de memoria en donde están guardadas.

El generador emite las instrucciones correspondientes para cada construcción del lenguaje. En nuestro diseño no adoptamos por completo las convenciones de la ABI de C; sin embargo, mantenemos la regla de que toda función devuelve su valor en el registro *RAX*. Cuando una función necesita preservar valores de registros cuando va a realizar un llamado a función, los guarda manualmente en la pila con *PUSH*, asumiendo que la función a la que llama

puede cambiar (casi) cualquier registro. Obviamente una función si deja el RSP como antes de ser llamada: cada función deja el stack de la misma manera que le llegó.

3.3. Adicionales

Para facilitar la validación de los programas compilados, desarrollamos dos scripts que automatizan el pipeline completo:

1. *microGCC.sh*: compila un archivo de prueba usando nuestro compilador, genera el *.asm*, lo ensambla y lo linkedita.
2. *microGCC_libasm.sh*: realiza el mismo proceso, pero además compila contra una librería auxiliar *libasm.asm* que contiene funciones predefinidas (como *print_number*, *print_str* o *init_array*).

Una vez generado el binario, se puede ejecutar el archivo con *./src/test/asm/output/output* y luego consultar su código de salida con *echo \$?*. Esto permite verificar de forma rápida y confiable si el comportamiento del programa fue el esperado. Para leer más sobre el funcionamiento de dichos scripts, se puede consultar el *README* del proyecto.

Asimismo, para facilitar la comprensión del programa y de su salida, se imprime la tabla de símbolos por la salida estándar (comentada en sintaxis de assembly). Esta tabla no se incorpora a los archivos ensamblador generados en */test/asm/output*, para no generar confusión en el usuario que solo le interese la salida del código generado (y no la implementación del backend).

3.4. Dificultades encontradas

Durante el desarrollo del proyecto nos encontramos con distintos tipos de dificultades en cada etapa del compilador. Una de las más importantes fue validar que el código assembler generado hiciera lo que realmente debía hacer. Al trabajar directamente con assembler x8086, no teníamos una forma intuitiva de saber si una función, un bucle o una asignación se estaban traduciendo correctamente. Para resolver esto, desarrollamos los scripts mencionados anteriormente. Esto facilitó mucho la comparación con los resultados esperados y nos permitió detectar errores de traducción, de manejo de registros o fallas en el uso del stack. Otra dificultad fue el diseño de la gramática debido a los conflictos de tipo *shift/reduce* o *reduce/reduce* previamente expuestos.

Por último, también fue necesario prestar atención al diseño y uso de la tabla de símbolos. La separación por scopes (globales y de funciones), el cálculo correcto de los offsets relativos a *RBP*, y el manejo diferenciado de

variables globales y locales en el código ensamblador, fueron problemas a resolver en el análisis semántico y la generación de código.

4. Futuras extensiones

Aunque el compilador ya genera código assembler funcional, hay varias cosas que se podrían mejorar o extender más adelante. En términos generales nos gustaría mejorar la eficiencia de los programas compilados. Para esto podríamos implementar algoritmos para asignar registros a las variables que más operaciones requieran. Ahora mismo, los usamos principalmente para almacenar valores temporales del programa, pero después las variables y los parámetros se consultan siempre desde el stack, que se encuentra en memoria lo que lleva a operaciones costosas. Con una mejor gestión de registros podríamos optimizar bastante el código generado y evitar operaciones innecesarias.

Otra mejora interesante sería agregar soporte para punteros y structs. Por ahora solo fueron considerados los números enteros y caracteres, pero incorporar punteros permitiría ampliar mucho las posibilidades del lenguaje, por ejemplo, para manejar estructuras de datos complejas y dinámicas.

5. Conclusiones

El desarrollo de un compilador que traduce un subconjunto del lenguaje C a código assembler para la arquitectura x86 representó una oportunidad concreta para aplicar e integrar múltiples conceptos centrales tanto de *Autómatas*, *Teoría de Lenguajes y Compiladores*, como de otras asignaturas, tanto como *Arquitectura de Computadoras* y *Sistemas Operativos*. A lo largo del proyecto se implementaron todas las etapas fundamentales de un compilador, desde el diseño de la gramática hasta la generación de código ejecutable.

El trabajo permitió comprender de manera concreta el funcionamiento interno de un compilador, los desafíos que implica cada etapa, y la importancia de mantener coherencia entre análisis sintáctico, semántico y generación de código. El resultado final no solo demuestra el funcionamiento del compilador, sino que también refleja un aprendizaje profundo sobre el diseño y la implementación de lenguajes de programación.

6. Referencias

- Intel Corporation. (n.d.). *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture* [PDF]. Retrieved June 2025, from <https://cdrdv2.intel.com/v1/dl/getContent/671110>